

CO2070 COMPUTER ARCHITECTURE

AHAMED M.K.H.

GROUP 06

E/22/014 , E/22/035

LAB 4_5

SEMESTER 4

18/06/2026

OVERVIEW

Lab 04_5 extends the single-cycle CPU designed in Lab 04 with six new instructions. The existing Verilog modules (pc.v, reg_file.v) remain unchanged. The ALU (alu.v) gains two new functional units, and the control unit inside cpu.v gains six new OPCODE decode entries and three new auxiliary control wires.

New Instructions at a Glance

Instructions highlighted in yellow are new additions in Lab 4_5.

Opcode	Mnemonic	Instruction Format	Operation	New
0x00	loadi	loadi RD, IMM	RD = IMM (8-bit immediate)	
0x01	mov	mov RD, RT	RD = RT	
0x02	add	add RD, RT, RS	RD = RT + RS	
0x03	sub	sub RD, RT, RS	RD = RT - RS	
0x04	and	and RD, RT, RS	RD = RT & RS	
0x05	or	or RD, RT, RS	RD = RT RS	
0x06	j	j OFFSET	PC = (PC+4) + sign_ext(OFFSET)*4	
0x07	beq	beq OFFSET, RT, RS	if RT==RS: PC = (PC+4)+offset	
0x08	mult	mult RD, RT, RS	RD = RT * RS (lower 8 bits)	Lab 4.5
0x09	sll	sll RD, RT, IMM	RD = RT << IMM (logical left)	Lab 4.5
0x0A	srl	srl RD, RT, IMM	RD = RT >> IMM (logical right)	Lab 4.5
0x0B	sra	sra RD, RT, IMM	RD = RT >>> IMM (arithmetic right)	Lab 4.5
0x0C	ror	ror RD, RT, IMM	RD = rotate_right (RT, IMM)	Lab 4.5
0x0D	bne	bne OFFSET, RT, RS	if RT!=RS: PC = (PC+4)+offset	Lab 4.5

CPU ARCHITECTURE CHANGES

The datapath is unchanged from Lab 04 except for the following additions:

ALU Extensions (alu.v)

- mult_unit — shift-and-add multiplier (SELECT = 3'b100)
- barrel_unit — iterative shift/rotate unit (SELECT = 3'b101), directed by SHIFT_DIR, ARITH, and ROTATE control lines
- ALU RESULT mux converted from always @(*) block to zero-delay continuous assign to prevent Verilog inertial-delay cancellation

Control Unit Extensions (cpu.v)

- Three new auxiliary control outputs: SHIFT_DIR, ARITH, ROTATE
- New branch signal: BRANCH_NE (used by bne)
- PC-select logic extended: PC_SEL = JUMP | (BRANCH & ZERO) | (BRANCH_NE & ~ZERO)
- Six new OPCODE case entries: 0x08–0x0D

Timing Budget (Single-Cycle, 8 ns Period)

All new instructions fit within the existing 8 time-unit budget:

Stage	Delay	Notes
PC Update	#1	pc.v, synchronous on posedge CLK
Instruction Fetch	#2	Testbench memory read
Decode (Control Unit)	#1	Combinational OPCODE → signals
Register File Read	#2	reg_file.v asynchronous read
ALU (mult / barrel)	#2	Transport delay on output wire
Register File Write	#1	reg_file.v synchronous write
Total	#7	Within 8 time-unit clock period

EXTENDED INSTRUCTION DOCUMENTATION

Each section below documents one new instruction: encoding, timing, datapath activation, worked example, and control signals.

1. mult — Integer Multiply

Syntax :

mult RD, RT, RS

Multiplies the 8-bit unsigned values in RT and RS, stores the lower 8 bits of the product in RD.

Instruction Encoding (32-bit word)

[31:24]	[23:16]	[15:8]	[7:0]
OPCODE	RD	RT	RS
0x08	RD addr	RT addr	RS addr

Timing Diagram

Clock period = 8 time units. Dark blue = active stage.

Stage	1	2	3	4	5	6	7	8
PC Update								
Instruction Fetch								
PC+4 Adder								
Decode								
Reg File Read								
mult_unit								
Reg File Write								

Worked Example :

mult r4, r1, r2 (where r1 = 3, r2 = 4)

r4 = r1 * r2 = 3 * 4 = 12 = 0x0C

mult_unit uses shift-and-add:

for each set bit i in $r2$, adds $(r1 \ll i)$ to a running partial sum. Overflow above 8 bits is discarded.

Control Signals :

Control Signal	Value	Reason
OPCODE	0x08	Selects mult decode path
ALUOP	3'b100	Routes ALU output from mult_unit
WRITEENABLE	1	Writes product back to RD
MUX_IMM_SEL	0	Register operands used (not immediate)
MUX_NEG_SEL	0	No two's complement needed
SHIFT_DIR	0	Don't-care (barrel_unit not selected)
ARITH	0	Don't-care (barrel_unit not selected)
ROTATE	0	Don't-care (barrel_unit not selected)

2. sll — Shift Left Logical

Syntax :

```
sll RD, RT, IMM
```

Shifts the value in RT left by IMM bit positions, filling vacated LSBs with 0. Stores result in RD.

Instruction Encoding

[31:24]	[23:16]	[15:8]	[7:0]
OPCODE	RD	RT	IMM
0x09	RD addr	RT addr	shift amount (0-7)

Timing Diagram

Stage	1	2	3	4	5	6	7	8
PC Update								
Instruction Fetch								
PC+4 Adder								
Decode								
Reg File Read								
barrel_unit (sll)								
Reg File Write								

Worked Example

```
sll r5, r1, 2 ( where r1 = 3 (0000 0011b) )
```

```
r5 = 3 << 2 = 0000 1100b = 0x0C = 12
```

Each shift-left by 1 discards the MSB and appends a 0 at the LSB. Repeating twice gives the result.

Control Signals

Control Signal	Value	Reason
OPCODE	0x09	Selects sll decode path
ALUOP	3'b101	Routes ALU output from barrel_unit
WRITEENABLE	1	Writes shifted value to RD
MUX_IMM_SEL	1	Shift amount comes from IMM field [7:0]
MUX_NEG_SEL	0	No two's complement
SHIFT_DIR	0	0 = left shift

ARITH	0	Logical only (left shift is always logical)
ROTATE	0	Shift, not rotate

3. srl — Shift Right Logical

Syntax

```
srl RD, RT, IMM
```

Shifts the value in RT right by IMM bit positions, filling vacated MSBs with 0. Stores result in RD.

Instruction Encoding

[31:24]	[23:16]	[15:8]	[7:0]
OPCODE	RD	RT	IMM
0x0A	RD addr	RT addr	shift amount (0-7)

Timing Diagram

Stage	1	2	3	4	5	6	7	8
PC Update								
Instruction Fetch								
PC+4 Adder								
Decode								
Reg File Read								
barrel_unit (srl)								
Reg File Write								

Worked Example

srl r6, r2, 1 where r2 = 4 (0000 0100b)

r6 = 4 >> 1 = 0000 0010b = 0x02 = 2

MSB is filled with 0 on each right-shift step. Contrast with sra which fills with the sign bit.

Control Signals

Control Signal	Value	Reason
OPCODE	0x0A	Selects srl decode path
ALUOP	3'b101	Routes ALU output from barrel_unit
WRITEENABLE	1	Writes shifted value to RD
MUX_IMM_SEL	1	Shift amount comes from IMM field [7:0]
MUX_NEG_SEL	0	No two's complement

SHIFT_DIR	1	1 = right shift
ARITH	0	0 = logical (zero fill from MSB)
ROTATE	0	Shift, not rotate

4. sra — Shift Right Arithmetic

Syntax

```
sra RD, RT, IMM
```

Shifts the value in RT right by IMM bit positions, filling vacated MSBs with the original sign bit (RT[7]). This preserves the two's-complement sign, effectively dividing a signed integer by 2^{IMM} . Stores result in RD.

Instruction Encoding

[31:24]	[23:16]	[15:8]	[7:0]
OPCODE	RD	RT	IMM
0x0B	RD addr	RT addr	shift amount (0-7)

Timing Diagram

Stage	1	2	3	4	5	6	7	8
PC Update								
Instruction Fetch								
PC+4 Adder								
Decode								
Reg File Read								
barrel_unit (sra)								
Reg File Write								

Worked Example

```
sra r7, r3, 2 ( where r3 = 0xFF (1111 1111b, sign bit = 1) )
```

```
r7 = 0xFF >>> 2 = 1111 1111b = 0xFF
```

Because the sign bit of r3 is 1, each right-shift step fills the new MSB with 1, preserving the negative value. Shifting 0xFF right arithmetically by any amount still yields 0xFF.

Control Signals

Control Signal	Value	Reason
OPCODE	0x0B	Selects sra decode path
ALUOP	3'b101	Routes ALU output from barrel_unit
WRITEENABLE	1	Writes shifted value to RD

MUX_IMM_SEL	1	Shift amount comes from IMM field [7:0]
MUX_NEG_SEL	0	No two's complement
SHIFT_DIR	1	1 = right shift
ARITH	1	1 = arithmetic (sign-bit fill from MSB)
ROTATE	0	Shift, not rotate

5. ror — Rotate Right

Syntax

```
ror RD, RT, IMM
```

Rotates the value in RT right by IMM bit positions. Bits that fall off the LSB wrap back into the MSB. No bits are lost. Stores result in RD.

Instruction Encoding

[31:24]	[23:16]	[15:8]	[7:0]
OPCODE	RD	RT	IMM
0x0C	RD addr	RT addr	rotate amount (0-7)

Timing Diagram

Stage	1	2	3	4	5	6	7	8
PC Update								
Instruction Fetch								
PC+4 Adder								
Decode								
Reg File Read								
barrel_unit (ror)								
Reg File Write								

Worked Example

ror r5, r0, 1 where r0 = 0x81 (1000 0001b)

Step 1: LSB (1) wraps to MSB position
1000 0001b → 1100 0000b = 0xC0

The LSB '1' of 0x81 rotates into the MSB position, giving 0xC0 = 1100 0000b.

Control Signals

Control Signal	Value	Reason
OPCODE	0x0C	Selects ror decode path
ALUOP	3'b101	Routes ALU output from barrel_unit
WRITEENABLE	1	Writes rotated value to RD
MUX_IMM_SEL	1	Rotate amount comes from IMM field [7:0]

MUX_NEG_SEL	0	No two's complement
SHIFT_DIR	1	Don't-care when ROTATE=1; set to 1 by convention
ARITH	0	Don't-care when ROTATE=1
ROTATE	1	1 = rotate right (overrides SHIFT_DIR)

6. bne — Branch if Not Equal

Syntax

```
bne OFFSET, RT, RS
```

Compares RT and RS. If they are NOT equal the program counter is updated to $(PC+4) + \text{sign_ext}(\text{OFFSET}) \times 4$. If they are equal execution continues sequentially. The comparison is performed by subtracting RS from RT using the existing add unit; WRITEENABLE = 0 so no register is modified.

Instruction Encoding

[31:24]	[23:16]	[15:8]	[7:0]
OPCODE	OFFSET	RT	RS
0x0D	signed offset	RT addr	RS addr

Timing Diagram

The branch target adder runs in parallel with the ALU subtract, both completing at cycle 7.

Stage	1	2	3	4	5	6	7	8
PC Update								
Instruction Fetch								
PC+4 Adder								
Decode								
2's Complement (RS)								
Reg File Read								
Branch/Jump Target Adder								
ALU (subtract → ZERO)								
PC Mux / Update								

Branch Target Calculation

Target = $(PC + 4) + \text{sign_ext}(\text{OFFSET}, 8 \rightarrow 32) \times 4$

OFFSET is stored in bits [23:16] of the instruction word (the RD_ADDR field).

Multiplying by 4 converts the instruction-count offset to a byte address offset, since each instruction occupies 4 bytes.

Worked Example (Branch Taken)

loadi r1, 5 → r1 = 5

loadi r2, 4 → r2 = 4

bne +1, r1, r2 ; r1(5) != r2(4), ZERO=0, branch TAKEN
target = (0x28 + 4) + 1*4 = 0x30

The instruction at 0x2C is skipped; execution resumes at 0x30.

Worked Example (Branch Not Taken)

loadi r1, 4 → r1 = 4 (same as r2)

bne +1, r1, r2 ; r1(4) == r2(4), ZERO=1, NOT taken

ZERO = 1, so BRANCH_NE does not fire. Next instruction executes sequentially.

PC Mux Logic

PC_SEL = JUMP | (BRANCH & ZERO) | (BRANCH_NE & ~ZERO)

For bne: BRANCH_NE = 1, BRANCH = 0, JUMP = 0. PC_SEL is HIGH only when ZERO = 0 (operands differ).

Control Signals

Control Signal	Value	Reason
OPCODE	0x0D	Selects bne decode path
ALUOP	3'b001	ALU performs subtraction to generate ZERO flag
WRITEENABLE	0	Registers not modified (compare only)
MUX_IMM_SEL	0	Register operands compared (not immediate)
MUX_NEG_SEL	1	RS is negated: ALU computes $RT + (-RS)$ = $RT - RS$
JUMP	0	Not an unconditional jump
BRANCH	0	Not a beq (ZERO=1) branch
BRANCH_NE	1	Branch taken when ZERO=0 ($RT \neq RS$)
SHIFT_DIR	0	Don't-care
ARITH	0	Don't-care
ROTATE	0	Don't-care

SIMULATION VERIFICATION

The testbench (cpu_tb.v) loads a 16-instruction program that exercises every new instruction plus both bne outcomes. All simulation results match the expected values.

Test Program Summary

#	Instruction	PC (hex)	Purpose
0	loadi r1, 3	0x00	Setup: r1 = 3
1	loadi r2, 4	0x04	Setup: r2 = 4
2	loadi r3, 0xFF	0x08	Setup: r3 = 0xFF (negative)
3	mult r4, r1, r2	0x0C	Test mult: $3 * 4 = 12$
4	sll r5, r1, 2	0x10	Test sll: $3 \ll 2 = 12$
5	srl r6, r2, 1	0x14	Test srl: $4 \gg 1 = 2$
6	sra r7, r3, 2	0x18	Test sra: $0xFF \ggg 2 = 0xFF$
7	loadi r0, 0x81	0x1C	Setup rotate input
8	ror r5, r0, 1	0x20	Test ror: <code>rotate_right(0x81,1)</code> = 0xC0
9	loadi r1, 5	0x24	Setup bne taken ($r1 \neq r2$)
10	bne +1, r1, r2	0x28	Test bne TAKEN → jump to 0x30
11	loadi r6, 0xDE	0x2C	SKIPPED by taken bne
12	loadi r6, 0xAA	0x30	Resumed after bne
13	loadi r1, 4	0x34	Setup bne not-taken ($r1 = r2$)
14	bne +1, r1, r2	0x38	Test bne NOT TAKEN → sequential
15	loadi r7, 0xBB	0x3C	Executes (bne not taken)

Final Register Verification

Simulation output (vvp sim) at t = 150 ns:

Register	Expected	Simulation	Test Instruction
r0	0x81	0x81	loadi r0, 0x81
r1	0x04	0x04	loadi r1, 4 (last write, instruction 13)
r2	0x04	0x04	loadi r2, 4
r3	0xFF	0xFF	loadi r3, 0xFF
r4	0x0C	0x0C	mult r4, r1, r2 $\rightarrow 3*4=12$
r5	0xC0	0xC0	ror r5, r0, 1 $\rightarrow$ rotate_right(0x81,1)
r6	0xAA	0xAA	loadi r6, 0xAA (0xDE skipped by bne)
r7	0xBB	0xBB	loadi r7, 0xBB (bne not taken, executes)

DELIVERABLE FILES

File	Changed?	Description
alu.v	Yes – extended	Added mult_unit, barrel_unit; mux converted to continuous assign
cpu.v	Yes – extended	New OPCODE cases 0x08–0x0D; SHIFT_DIR, ARITH, ROTATE, BRANCH_NE wires
cpu_tb.v	Yes – new	16-instruction test program covering all 6 new instructions
reg_file.v	No – unchanged	8-register synchronous write, asynchronous read (Lab 02)
pc.v	No – unchanged	Program counter with synchronous reset (Lab 03)